

Gerência de Configurações e Automação

Fundamentos, IaC e Ferramentas Essenciais para Infraestrutura Moderna

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Sumário

- Fundamentos e Conceitos de Gerência de Configurações
- Infraestrutura como Código (IaC) e Benefícios
- Provisionamento com Terraform e Configuração com Ansible
- Puppet e Vagrant: Alternativas para Automação
- Comparativo de Ferramentas e Discussão Prática

INSTITUIÇÃO EXECUTORA



COORDENADORA



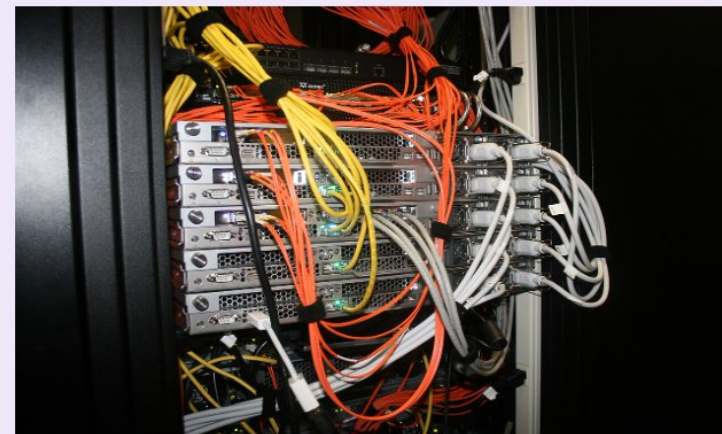
APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Fundamentos da Gerência de Configurações

A **Gerência de Configurações (GC)** é uma disciplina da engenharia de software e infraestrutura essencial para a manutenção da consistência de um produto ao longo do seu ciclo de vida. Em ambientes modernos, a GC resolve o problema do '*Configuration Drift*' (deriva de configuração), que ocorre quando servidores que deveriam ser idênticos divergem devido a intervenções manuais. Os objetivos principais da GC incluem: **Consistência**, garantindo que todos os servidores de um cluster possuam as mesmas versões de pacotes e configurações; **Repetibilidade**, permitindo recriar todo um ambiente do zero de forma idêntica; **Integridade**, oferecendo controle total sobre quem alterou o quê e quando; e **Automação**, reduzindo o erro humano ao substituir scripts ad-hoc por definições declarativas.



Gerenciamento de cabos em racks de servidores da NASA: consistência e automação.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Pilares Fundamentais da Gerência de Configuração



Idempotência em Operações

A **idempotência** é uma propriedade crítica que permite que uma operação seja executada várias vezes, sempre resultando no mesmo estado final que seria alcançado com apenas uma execução inicial. Isso significa que, se o sistema já se encontra no estado desejado, a ferramenta de Gerência de Configuração (GC) não realizará nenhuma ação, evitando operações redundantes e potencialmente disruptivas. Este princípio é fundamental para a estabilidade e previsibilidade em ambientes automatizados.



Estado Desejado vs. Atual

As ferramentas de GC operam monitorando o *estado atual* de um sistema, comparando-o com o *estado desejado* definido por código. Elas aplicam apenas as modificações estritamente necessárias para que o sistema real (estado atual) se alinhe com a configuração especificada (estado desejado). Esta abordagem declarativa garante que a infraestrutura esteja sempre em conformidade com as diretrizes definidas, minimizando desvios de configuração.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Pilares Fundamentais da Gerência de Configuração



Inventário de Ativos

O *inventário* é uma lista abrangente de todos os nós (servidores físicos ou virtuais, instâncias em nuvem, dispositivos de rede, etc.) que estão sob a gestão da ferramenta de GC. É comumente organizado em grupos lógicos, como `[webservers]`, `[databases]` ou `[firewall_rules]`, permitindo a aplicação de configurações específicas a conjuntos de sistemas. Um inventário bem estruturado é vital para a aplicação eficiente e escalável das políticas de automação.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Infraestrutura como Código (IaC)

A **Infraestrutura como Código (IaC)** é uma abordagem para gerenciar a infraestrutura (como redes, máquinas virtuais e balanceadores de carga) por meio de arquivos de definição legíveis por máquina, em vez de configuração manual. Isso permite que a infraestrutura seja tratada como software, possibilitando vários benefícios:

- **Versionamento:** O código da infraestrutura é armazenado em sistemas de controle de versão (como Git), facilitando o rastreamento, a revisão e o *rollback* de alterações, garantindo auditabilidade e controle. Cada mudança é registrada e pode ser revertida, se necessário.
- **Velocidade:** Ambientes complexos podem ser provisionados e configurados em questão de minutos, eliminando processos manuais demorados e suscetíveis a erros.
- **Documentação Implícita:** O próprio código atua como documentação precisa e atualizada da infraestrutura, eliminando a necessidade de manter documentos separados que podem se tornar desatualizados.
- **Escalabilidade:** A IaC permite a criação e gerenciamento de múltiplas instâncias de infraestrutura de forma automatizada e consistente, sem exigir esforço manual adicional para cada nova instância.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Provisionamento vs. Gerência de Configuração

Embora os termos se sobreponham, há uma distinção arquitetural importante entre as ferramentas de Provisionamento e Gerência de Configuração.

Atributo	Provisionamento (Ex: Terraform)	Gerência de Configuração (Ex: Ansible)
Foco Principal	Criar a infraestrutura 'vazia' (VMs, VPCs, Subnets).	Configurar o software dentro da infraestrutura (Nginx, MySQL).
Nível de Abstração	Recursos de Nuvem / Datacenter.	Sistema Operacional / Aplicação.
Modelo	Declarativo (Estado final).	Pode ser Híbrido (Procedural + Declarativo).
Exemplo de Ação	'Crie 3 instâncias AWS EC2'.	'Instale e inicie o Apache'.



INSTITUIÇÃO EXECUTORA



COORDENADORA



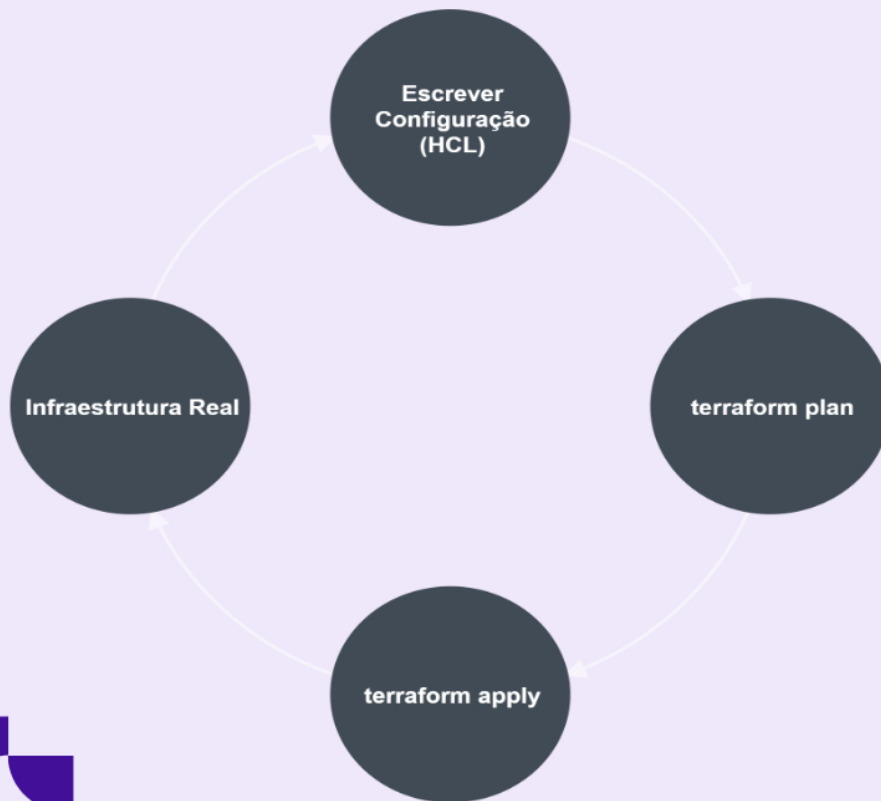
APOIO



Terraform: Orquestração de Recursos

Base do Terraform

O Terraform é uma ferramenta de provisionamento de infraestrutura como Código (IaC), *agnóstica de nuvem*, o que significa que pode gerenciar recursos em diversos provedores. Ele utiliza a linguagem HCL (HashiCorp Configuration Language) para descrever e orquestrar recursos de infraestrutura de forma declarativa, permitindo que os desenvolvedores definam o estado desejado da infraestrutura.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Terraform: Orquestração de Recursos

Plano e Estado

O comando "terraform plan" é crucial, pois notificou as mudanças que serão aplicadas, mostrando um resumo detalhado antes de qualquer alteração ser efetuada. O arquivo "terraform.tfstate" é o coração do gerenciador de estado do Terraform; ele mapeia os recursos reais provisionados para a configuração definida, garantindo que os recursos reais sejam mantidos.

Provedores e Grafos

Os *Provedores* (Providers) são módulos de software que permitem ao Terraform interagir com APIs de diversos serviços, como AWS, Azure, Google Cloud ou VMware, para criar, modificar e excluir recursos. O Terraform constrói automaticamente um *grafo de recursos* (dependency graph), um mapa de dependências que assegura que os recursos sejam criados ou destruídos na ordem lógica correta, evitando erros e garantindo a consistência da infraestrutura.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Ansible: Automação Simples e Direta

- O Ansible se destaca por ser '**agentless**', o que significa que não requer software instalado nos nós gerenciados, utilizando protocolos padrão como SSH para Linux/Unix ou WinRM para Windows.
- Os **Playbooks** são arquivos YAML que descrevem as tarefas a serem executadas e o estado desejado dos sistemas de forma declarativa, facilitando a leitura e a manutenção.
- Os **Módulos** são unidades de código reutilizáveis que realizam funções específicas, como instalar pacotes, gerenciar serviços, copiar arquivos ou configurar usuários, tornando a automação flexível.
- Utiliza um **Modelo Push**, onde o nó controlador envia diretamente as configurações e comandos para os nós remotos, garantindo que as máquinas-alvo recebam as instruções necessárias.
- Sua **sintaxe amigável**, baseada em YAML, simplifica a curva de aprendizado para desenvolvedores e administradores, permitindo que a automação seja escrita e compreendida com maior facilidade.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Puppet: Modelo Baseado em Estado

Funcionamento 'Pull'

O Puppet é uma das ferramentas de Gerência de Configurações (GC) mais estabelecidas. Diferente do Ansible, ele adota tradicionalmente um modelo *Pull*, onde agentes (Puppet Agent) são instalados em cada nó cliente. Esses agentes se comunicam com o Puppet Server para obter as configurações e aplicá-las.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Puppet: Modelo Baseado em Estado

Linguagem e Manifestos

O Puppet utiliza uma *DSL* (Domain Specific Language — Linguagem de Domínio Específico) própria, baseada em Ruby, para descrever o estado desejado dos sistemas. As configurações são declaradas em arquivos chamados **Manifestos**, onde são definidos recursos como pacotes a serem instalados, arquivos a serem configurados e usuários a serem criados.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Puppet: Modelo Baseado em Estado

Agente e Catálogo

O **Puppet Agent** executa periodicamente (geralmente a cada 30 minutos) para verificar e garantir que o estado atual do sistema não se desviou do estado definido. O **Puppet Server** é responsável por compilar os manifestos em um **Catálogo** específico para cada agente. Este catálogo é um documento que lista todos os recursos e suas configurações, garantindo que a aplicação seja feita de forma consistente e localmente pelo agente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Vagrant: Ambientes de Desenvolvimento



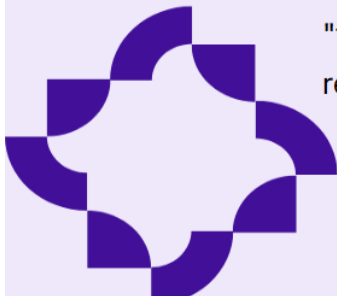
Padronização com Vagrant

O Vagrant é uma ferramenta crucial para construir e gerenciar ambientes de desenvolvimento virtualizados e portáteis. Seu principal objetivo é garantir que o ambiente de trabalho no computador de cada desenvolvedor seja **idêntico** ao de seus colegas e o mais próximo possível do ambiente de produção. Isso elimina o famoso problema "funciona na minha máquina", promovendo consistência e reduzindo erros de configuração.



Base de Imagens: Boxes

Os *Boxes* são imagens base de sistemas operacionais pré-configuradas, como Ubuntu, CentOS ou Debian, que servem como ponto de partida para as máquinas virtuais do Vagrant. Ao invés de instalar um SO do zero, o desenvolvedor escolhe um box, e o Vagrant o utiliza para criar uma VM rapidamente. Isso garante que todos os ambientes comecem da mesma base.



INSTITUIÇÃO EXECUTORA



COORDENADORA

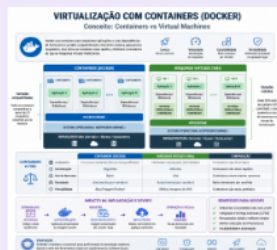


APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Vagrant: Ambientes de Desenvolvimento



Gerenciamento de Providers

Os *Providers* são os softwares de virtualização que o Vagrant gerencia para criar e executar as máquinas virtuais. Exemplos incluem VirtualBox, VMware, Hyper-V e Docker. O Vagrant abstrai as particularidades de cada provider, permitindo que o desenvolvedor use o mesmo fluxo de trabalho independentemente da tecnologia de virtualização subjacente.

Portabilidade e Compartilhamento

Uma das grandes vantagens do Vagrant é sua **portabilidade**. Para compartilhar um ambiente de desenvolvimento completo, basta distribuir um único arquivo de configuração chamado ``Vagrantfile``. Qualquer outro desenvolvedor com Vagrant instalado pode usar este arquivo para provisionar e subir a mesma máquina virtual, com as mesmas configurações e dependências, em minutos.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Vagrant: Ambientes de Desenvolvimento

Automação de Provisionamento

O Vagrant integra-se com ferramentas de automação para **provisionamento** do ambiente. Após o boot da máquina virtual, ele pode chamar automaticamente *Ansible*, *Puppet*, *Chef* ou *Shell Scripts* para instalar softwares, configurar serviços, criar diretórios e aplicar quaisquer outras configurações necessárias, garantindo que a VM esteja pronta para o desenvolvimento imediatamente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Ciclo de Trabalho do Vagrant

1

Inicializar o Projeto

O comando ``vagrant init`` é o ponto de partida, criando um arquivo de configuração chamado ``Vagrantfile`` no diretório atual. Este arquivo define as especificações da máquina virtual, como o sistema operacional base (box) e configurações de rede.

2

Levantar a Máquina Virtual

Com ``vagrant up``, o Vagrant provisiona e configura a máquina virtual de acordo com o ``Vagrantfile``. Se provisionadores como scripts shell ou ferramentas de gerência de configuração estiverem definidos, eles são executados nesta etapa, garantindo um ambiente pronto para uso.

3

Acessar via SSH

Utilize ``vagrant ssh`` para estabelecer uma conexão segura (SSH) diretamente com a máquina virtual. Este comando simplifica o acesso, eliminando a necessidade de gerenciar chaves SSH ou endereços IP manualmente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Ciclo de Trabalho do Vagrant

4

Desligar a Máquina Virtual

O comando ``vagrant halt`` desliga a máquina virtual de forma controlada. O estado atual da VM é preservado, permitindo que ela seja reiniciada rapidamente mais tarde a partir do ponto onde parou.

5

Reiniciar e Reaplicar Configurações

Se houver alterações no ``Vagrantfile``, como novas pastas sincronizadas ou provisionadores atualizados, ``vagrant reload`` é usado para reiniciar a máquina virtual e aplicar essas novas configurações.

6

Destruir a Máquina Virtual

Para remover completamente a máquina virtual e todos os seus recursos (disco, memória, configurações) do sistema hospedeiro, utilize ``vagrant destroy``. Este comando garante uma limpeza total do ambiente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Comparativo de Ferramentas de GC

Esta tabela resume as principais diferenças entre as ferramentas estudadas para auxiliar na escolha da solução ideal por projeto.

Ferramenta	Modelo	Linguagem	Agente	Melhor Uso
Ansible	Push	YAML	Não	Automação rápida, deploy de apps, orquestração ad-hoc.
Puppet	Pull	DSL (Ruby)	Sim	Grandes parques de servidores, conformidade de longo prazo.
Terraform	Declarativo	HCL	Não	Provisionamento de recursos em nuvem e rede.
Vagrant	Local	Ruby	N/A	Padronização de ambiente de dev local.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Considerando os conceitos e as ferramentas de Gerência de Configurações e Automação que revisamos, qual das seguintes afirmações sobre esses princípios e suas ferramentas está ***correta***?

- A Idempotência significa que aplicar a mesma configuração múltiplas vezes não deve causar efeitos colaterais indesejados após a primeira aplicação.
- B O Ansible exige a instalação de um agente pesado em cada nó gerenciado para sua operação.
- C O Vagrant é utilizado exclusivamente para gerenciar servidores em produção na nuvem AWS, sem outras aplicações.
- D O Terraform é focado principalmente na configuração de software dentro do sistema operacional, enquanto o Ansible foca na criação de hardware virtual.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Considerando os conceitos e as ferramentas de Gerência de Configurações e Automação que revisamos, qual das seguintes afirmações sobre esses princípios e suas ferramentas está ***correta***?

- ✓ Idempotência significa que aplicar a mesma configuração múltiplas vezes não deve causar efeitos colaterais indesejados após a primeira aplicação.
- ✗ O Ansible exige a instalação de um agente pesado em cada nó gerenciado para sua operação.
- ✗ O Vagrant é utilizado exclusivamente para gerenciar servidores em produção na nuvem AWS, sem outras aplicações.

✗ O Terraform é focado principalmente na configuração de software dentro do sistema operacional, enquanto o Ansible foca na criação de hardware virtual.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Discuta com o colega ao lado

Como você abordaria este cenário de startup, combinando as ferramentas estudadas e os princípios de Infraestrutura como Código (IaC)?



Considere a criação de clusters Kubernetes, gerenciamento de bancos de dados e garantia de ambientes de desenvolvimento local consistentes.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Conclusão

- Gerência de Configurações mantém consistência, repetibilidade e integridade, combatendo a deriva de configuração em sistemas complexos.
- IaC (Infraestrutura como Código) com ferramentas como Terraform, Ansible e Puppet, versiona e automatiza a infraestrutura.
- Terraform orquestra recursos declarativamente em nuvem, utilizando HCL, planos e provedores para gerenciar o estado da infraestrutura.
- Ansible e Puppet configuram software: Ansible é agentless e push (YAML), Puppet usa agentes pull (DSL) para controle contínuo.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

